

一、实习时间

2026年6月23日至2026年6月26日

二、实习地点

C209

三、实习学时

25

四、实习目的

掌握物联网网络通信基本原理，熟悉MQTT协议，学习网络协议分析方法，提高Java网络程序设计与实践能力。

五、实习内容

搭建MQTT实验环境，完成消息发布与订阅，分析MQTT报文，编写TCP、UDP及简易MQTT客户端程序。

六、实习要求

熟悉实验内容，规范完成各项实验操作，正确分析实验结果，按要求完成程序设计及实习报告。

七、实验主要设备

计算机、EMQX服务器、MQTTX客户端、Wireshark抓包工具、Java开发环境及实验网络环境。

八、实验成果

完成MQTT协议分析及网络程序开发，实现客户端通信功能，掌握协议分析方法并完成实习报告。

1.1 系统概述

本实验基于 MQTT 协议 3.1.1 版本，搭建物联网消息通信实验环境，利用 EMQX 作为消息代理服务器，MQTTX 作为可视化客户端工具，Wireshark 作为网络抓包分析工具，对 MQTT 协议的 14 种报文格式进行捕获与分析，并通过 Python 编程实现 MQTT 客户端的发布与订阅功能。MQTT 是一种基于发布/订阅模式的轻量级消息传输协议，广泛应用于物联网场景中设备与服务器之间的通信。

1.2 需求分析

随着物联网技术的快速发展，大量传感器设备需要在带宽有限、网络不稳定的环境下进行可靠通信。MQTT 协议以其轻量、低带宽占用、支持不同服务质量等级（QoS）的特点，成为物联网通信领域的主流协议之一。通过对 MQTT 协议的深入学习与分析，能够掌握物联网应用层协议的设计原理，理解发布/订阅通信模式，为今后从事物联网应用开发奠定基础。

1.3 运行环境

- 操作系统：macOS Sequoia（Apple M3 芯片）
- MQTT Broker：EMQX 5.8.8（通过 Homebrew 安装，运行于本机 127.0.0.1:1883）
- MQTT 客户端：MQTTX Desktop
- 抓包工具：Wireshark 4.6.6
- 编程语言：Python 3.9.6，依赖库 paho-mqtt 2.1.0
- 协议版本：MQTT 3.1.1

2.MQTT协议分析

2.1 实验准备

实验环境：使用 Homebrew 在本机安装并启动 EMQX 5.8.8 作为 MQTT Broker，监听端口 1883。安装 MQTTX 作为 MQTT 客户端，Wireshark 用于网络流量抓包分析。

客户端配置：

- Client ID：client2411630201
- 连接地址：127.0.0.1:1883
- 协议版本：MQTT 3.1.1
- Keep Alive：60秒
- 订阅主题：小鱼（QoS 0）、xiaoyu（QoS 0）

抓包配置：Wireshark 选择 Loopback: lo0 接口进行抓包，显示过滤器设置为 mqtt，只显示 MQTT 协议报文。

2.2 主要实验步骤及14种报文分析

1. CONNECT（连接请求）

触发操作：MQTTX 点击 Connect 按钮连接 EMQX。

字段含义：

- Message Type (1)：报文类型，值为1代表 CONNECT
- Protocol Name：协议名称，值为 MQTT
- Protocol Version：协议版本，值为4代表 MQTT 3.1.1

- **Connect Flags:** 连接标志位, Clean Session=1 表示不保留历史会话
- **Keep Alive:** 心跳间隔, 值为60秒, 超时未通信则断开
- **Client ID:** 客户端唯一标识, 值为 client2411630201

> Frame 47: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface 0	0000	02 00 00 00 45 00 00 52 00 00 40 00 00 06 00 00	E..R..@..
> Null/Loopback	0010	7f 00 00 01 7f 00 00 01 c0 fd 07 5b 5f 5d 71 d5	[.]q..
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	71 ed a4 ce 80 18 18 ec fe 46 00 00 01 01 08 0a	F.....
> Transmission Control Protocol, Src Port: 49405, Dst Port: 1883, Seq: 363330323031	0030	81 0e d3 80 d0 0f 05 e6 10 1c 00 04 4d 51 54 54	MQTT...
> MQ Telemetry Transport Protocol, Connect Command	0040	04 02 00 3c 00 10 63 6c 69 65 6e 74 32 34 31 31	...<..cl ient2411
> Header Flags: 0x10, Message Type: Connect Command	0050	36 33 30 32 30 31	630201
Msg Len: 28			
Protocol Name Length: 4			
Protocol Name: MQTT			
Version: MQTT v3.1.1 (4)			
Connect Flags: 0x02, QoS Level: At most once delivery (Fire and Forget)			
Keep Alive: 60			
Client ID Length: 16			
Client ID: client2411630201			

2. CONNACK (连接确认)

触发操作: EMQX 收到 CONNECT 后回复。

字段含义:

- **Message Type (2):** 报文类型, 值为2代表 CONNACK
- **Session Present:** 会话标志, 0表示全新会话
- **Return Code:** 返回码, 0表示连接成功, 非0表示失败原因

> Frame 49: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0	0000	02 00 00 00 45 00 00 38 00 00 40 00 00 06 00 00	E..8..@..
> Null/Loopback	0010	7f 00 00 01 7f 00 00 01 07 5b c0 fd 71 ed a4 ce	[.]q..
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	5f 5d 71 f3 80 18 18 ec fe 2c 00 00 01 01 08 0a	,.....
> Transmission Control Protocol, Src Port: 1883, Dst Port: 49405, Seq: 20020000	0030	d0 0f 05 ea 81 0e d3 80 20 02 00 00	
> MQ Telemetry Transport Protocol, Connect Ack			
> Header Flags: 0x20, Message Type: Connect Ack			
Msg Len: 2			
Acknowledge Flags: 0x00			
Return Code: Connection Accepted (0)			

3. SUBSCRIBE (订阅请求)

触发操作: MQTTX 点击 New Subscription, 订阅主题 小鱼 和 xiaoyu。

字段含义:

- **Message Type (8):** 报文类型, 值为8代表 SUBSCRIBE
- **Message ID:** 消息标识符, 用于匹配 SUBACK
- **Topic:** 订阅的主题名称
- **QoS:** 请求的服务质量等级, 本实验设置为0

> Frame 914: Packet, 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface 0	0000	02 00 00 00 45 00 00 41 00 00 40 00 00 06 00 00	E..A..@..
> Null/Loopback	0010	7f 00 00 01 7f 00 00 01 c1 23 07 5b 88 c7 1e ab	#..[.]..
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	90 eb 7e b4 80 18 18 ec fe 35 00 00 01 01 08 0a	5.....
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq: 8f e9 b1 bc 00	0030	f0 cc 2f 40 38 1b 81 81 82 0b 46 66 00 06 e5 b0	.../@8...Ff..
> MQ Telemetry Transport Protocol, Subscribe Request	0040	8f e9 b1 bc 00	
> Header Flags: 0x82, Message Type: Subscribe Request			
1000 = Message Type: Subscribe Request (8)			
.... 0010 = Reserved: 2			
Msg Len: 11			
Message Identifier: 18022			
Topic Length: 6			
Topic: 小鱼			
Requested QoS: At most once delivery (Fire and Forget) (0)			

4. SUBACK (订阅确认)

触发操作: EMQX 确认订阅请求后回复。

字段含义:

- **Message Type (9):** 报文类型，值为9代表 SUBACK
- **Message ID:** 与 SUBSCRIBE 的 ID 对应
- **Return Code:** 订阅结果，0x00表示 QoS 0 订阅成功

> Frame 916: Packet, 61 bytes on wire (488 bits), 61 bytes captured (488 bits) on interface 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 1883, Dst Port: 49443, Seq: 300000000, Win: 65535, Len: 0
MQ Telemetry Transport Protocol, Subscribe Ack
Header Flags: 0x90, Message Type: Subscribe Ack
1001 = Message Type: Subscribe Ack (9)
.... 0000 = Reserved: 0
Msg Len: 3
Message Identifier: 18022
Granted QoS: At most once delivery (Fire and Forget) (0)

0000 02 00 00 00 45 00 00 39 00 00 40 00 40 06 00 00E..9..@..@..
0010 7f 00 00 01 7f 00 00 01 07 5b c1 23 90 eb 7e b4#..~..
0020 88 c7 1e b8 80 18 18 ec fe 2d 00 00 01 01 08 0aF..8..
0030 38 1b ca 0c f0 cc 2f 40 90 03 46 66 00/@..Ff..

5. PUBLISH (发布消息)

触发操作：MQTTX 向主题 小鱼 发送消息。

字段含义：

- **Message Type (3):** 报文类型，值为3代表 PUBLISH
- **DUP Flag:** 重复标志，1表示该消息是重发
- **QoS Level:** 服务质量等级，0/1/2
- **Retain:** 保留标志，1表示服务器保留该消息
- **Topic:** 发布的主题名称
- **Message ID:** 仅 QoS 1/2 时存在，用于确认
- **Payload:** 消息内容

(1) QoS 0

> Frame 2493: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq: 300000000, Win: 65535, Len: 0
MQ Telemetry Transport Protocol, Publish Message
Header Flags: 0x30, Message Type: Publish Message, QoS Level: At most once delivery (Fire and Forget) (0)
0011 = Message Type: Publish Message (3)
.... 0... = DUP Flag: Not set
.... 00. = QoS Level: At most once delivery (Fire and Forget)
.... ...0 = Retain: Not set
Msg Len: 28
Topic Length: 6
Topic: 小鱼
Message: 7b0a2020226d7367223a202268656c6c6f220a7d

0000 02 00 00 00 45 00 00 52 00 00 40 00 40 06 00 00E..R..@..@..
0010 7f 00 00 01 7f 00 00 01 c1 23 07 5b 88 c7 1e c9#..[..
0020 90 eb 7e c2 80 18 18 ec fe 46 00 00 01 01 08 0aF..8..
0030 f0 ce 2d 12 38 1d b9 b6 30 1c 00 00 e5 b0 8f e98...0...
0040 b1 bc 7b 0a 20 20 22 6d 73 67 22 3a 20 22 68 65 ...{"m sg": "he
0050 6c 6c 6f 22 0a 7dllo".}

Topic: 小鱼 QoS: 0					
{					
"msg": "hello"					
}					
2026-06-23 16:29:28:107					
Topic: 小鱼 QoS: 0					
{					
"msg": "hello"					
}					
2026-06-23 16:29:28:114					
2493	228.373289	127.0.0.1	127.0.0.1	MQTT	86 Publish Message [小鱼]
2495	228.374097	127.0.0.1	127.0.0.1	MQTT	86 Publish Message [小鱼]

2026-06-23 16:29:20-1

Topic: xiaoyu QoS: 0

```
{
  "msg": "HELLO"
}
```

2026-06-23 16:29:57:768

> Frame 2733: Packet, 86 bytes on wire (688 bits), 86 bytes captured (		0000	02 00 00 00 45 00 00 52	00 00 40 00 00 06 00 00	E..R..@.@...
> Null/Loopback		0010	7f 00 00 01 7f 00 00 01	c1 23 07 5b 88 c7 1e e7	#.[.....
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1		0020	90 eb 7e e0 80 18 18 ec	fe 46 00 00 01 01 08 0a	I.....
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq:		0030	f0 ce a0 ef 38 1d c7 dd	30 1c 00 06 78 69 61 6f	...x8!..? 2.....
> MQ Telemetry Transport Protocol, Publish Message		0040	79 75 7b 0a 20 20 22 6d	73 67 22 3a 20 22 48 45	..Fh{"msg": "HE
Header Flags: 0x30, Message Type: Publish Message, QoS Level: At		0050	4c 4c 4f 22 0a 7d		ciallo". }
0011 = Message Type: Publish Message (3)					
.... 0... = DUP Flag: Not set					
.... .00. = QoS Level: At most once delivery (Fire and Forget)					
.... ...0 = Retain: Not set					
Msg Len: 28					
Topic Length: 6					
Topic: xiaoyu					
Message: 7b0a2020226d7367223a202248454c4c4f220a7d					
2733	258.034825	127.0.0.1	127.0.0.1	MQTT	86 Publish Message [xiaoyu]

(2) QoS 1

> Frame 4997: Packet, 89 bytes on wire (712 bits), 89 bytes captured (		0000	02 00 00 00 45 00 00 55	00 00 40 00 00 06 00 00	E..U..@.@...
> Null/Loopback		0010	7f 00 00 01 7f 00 00 01	c1 23 07 5b 88 c7 1f 0d	#.[.....
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1		0020	90 eb 7e e0 80 18 18 ec	fe 49 00 00 01 01 08 0a	I.....
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq:		0030	f0 d3 0d 78 38 21 e5 3f	32 1f 00 06 e5 b0 8f e9	...x8!..? 2.....
> MQ Telemetry Transport Protocol, Publish Message		0040	b1 bc 46 68 7b 0a 20 20	22 6d 73 67 22 3a 20 22	..Fh{"msg": "
Header Flags: 0x32, Message Type: Publish Message, QoS Level: At		0050	63 69 61 6c 6c 6f 22 0a	7d	ciallo". }
0011 = Message Type: Publish Message (3)					
.... 0... = DUP Flag: Not set					
.... .01. = QoS Level: At least once delivery (Acknowledged del					
.... ...0 = Retain: Not set					
Msg Len: 31					
Topic Length: 6					
Topic: 小鱼					
Message Identifier: 18024					
Message: 7b0a2020226d7367223a20226369616c6c6f220a7d					

Topic: 小鱼 QoS: 1

```
{
  "msg": "ciallo"
}
```

2026-06-23 16:34:47:706

Topic: 小鱼 QoS: 0

```
{
  "msg": "ciallo"
}
```

2026-06-23 16:34:47:711

4997	547.968054	127.0.0.1	127.0.0.1	MQTT	89 Publish Message (id=18024) [小鱼]
4999	547.968851	127.0.0.1	127.0.0.1	MQTT	60 Publish Ack (id=18024)
5001	547.969016	127.0.0.1	127.0.0.1	MQTT	87 Publish Message [小鱼]

(3) QoS 2

Topic: 小鱼 QoS: 0

```
{
  "msg": "hi"
}
```

2026-06-23 16:36:40:208

Topic: 小鱼 QoS: 2

```
{
  "msg": "hi"
}
```

2026-06-23 16:36:40:212

```
> Frame 5961: Packet, 85 bytes on wire (680 bits), 85 bytes captured (
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq:
MQ Telemetry Transport Protocol, Publish Message
  Header Flags: 0x34, Message Type: Publish Message, QoS Level: Exa
    0011 .... = Message Type: Publish Message (3)
    .... 0... = DUP Flag: Not set
    .... 10.. = QoS Level: Exactly once delivery (Assured Delivery)
    .... ...0 = Retain: Not set
  Msg Len: 27
  Topic Length: 6
  Topic: 小鱼
  Message Identifier: 18025
  Message: 7b0a2020226d7367223a202268692220a7d
0000 02 00 00 00 45 00 00 51 00 00 40 00 40 06 00 00  ....E..Q..@.@...
0010 7f 00 00 01 7f 00 00 01 c1 23 07 5b 88 c7 1f 30  ....#.[...0
0020 90 eb 7f 0d 80 18 18 ec fe 45 00 00 01 01 08 0a  ....E.....
0030 f0 d4 c4 e0 38 23 92 a9 34 1b 00 06 e5 b0 8f e9  ....8#..4.....
0040 b1 bc 46 69 7b 0a 20 20 22 6d 73 67 22 3a 20 22  ..Fi{.. "msg": "
0050 68 69 22 0a 7d                                     hi"}.
```

5961	660.466290	127.0.0.1	127.0.0.1	MQTT	85	Publish Message (id=18025) [小鱼]
5963	660.468197	127.0.0.1	127.0.0.1	MQTT	60	Publish Received (id=18025)
5965	660.468298	127.0.0.1	127.0.0.1	MQTT	83	Publish Message [小鱼]
5967	660.471465	127.0.0.1	127.0.0.1	MQTT	60	Publish Release (id=18025)
5969	660.471785	127.0.0.1	127.0.0.1	MQTT	60	Publish Complete (id=18025)

6. PUBACK（发布确认，QoS 1）

触发操作：MQTTX 以 QoS 1 发布消息后，EMQX 回复确认。

字段含义：

- Message Type (4)：报文类型，值为4代表 PUBACK
- Message ID：与对应 PUBLISH 的 ID 一致，确认该消息已收到

7. PUBREC（发布收到，QoS 2 第一步）

触发操作：MQTTX 以 QoS 2 发布消息，EMQX 回复第一步确认。

字段含义：

- Message Type (5)：报文类型，值为5代表 PUBREC
- Message ID：与 PUBLISH 的 ID 对应，表示已收到消息

8. PUBREL（发布释放，QoS 2 第二步）

触发操作：客户端收到 PUBREC 后回复。

字段含义：

- Message Type (6)：报文类型，值为6代表 PUBREL
- Message ID：与 PUBREC 的 ID 对应，通知服务器可以投递消息了

9. PUBCOMP（发布完成，QoS 2 第三步）

触发操作：EMQX 收到 PUBREL 后回复，完成 QoS 2 四步握手。

字段含义：

- Message Type (7)：报文类型，值为7代表 PUBCOMP

- **Message ID:** 与 PUBREL 的 ID 对应，表示 QoS 2 流程完成

10. UNSUBSCRIBE (取消订阅)

触发操作：MQTTX 删除订阅主题 小鱼 和 xiaoyu。

字段含义：

- **Message Type (10):** 报文类型，值为10代表 UNSUBSCRIBE
- **Message ID:** 用于匹配 UNSUBACK
- **Topic:** 要取消订阅的主题名称

```
> Frame 6756: Packet, 68 bytes on wire (544 bits), 68 bytes captured (
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq:
MQ Telemetry Transport Protocol, Unsubscribe Request
  Header Flags: 0xa2, Message Type: Unsubscribe Request
    1010 .... = Message Type: Unsubscribe Request (10)
    .... 0010 = Reserved: 2
  Msg Len: 10
  Message Identifier: 18026
  Topic Length: 6
  Topic: 小鱼
0000 02 00 00 00 45 00 00 40 00 00 40 00 00 06 00 00  ....E..@..@...
0010 7f 00 00 01 7f 00 00 01 c1 23 07 5b 88 c7 1f 53  .......#.[...S
0020 90 eb 7f 32 80 18 18 ec fe 34 00 00 01 01 08 0a  ...2....4.....
0030 f0 d5 bc 40 38 25 4a 13 a2 0a 46 6a 00 06 e5 b0  ..@8%J...Fj....
0040 8f e9 b1 bc                                     ....
```

11. UNSUBACK (取消订阅确认)

触发操作：EMQX 确认取消订阅后回复。

字段含义：

- **Message Type (11):** 报文类型，值为11代表 UNSUBACK
- **Message ID:** 与 UNSUBSCRIBE 的 ID 对应

```
> Frame 6758: Packet, 60 bytes on wire (480 bits), 60 bytes captured (
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 1883, Dst Port: 49443, Seq:
MQ Telemetry Transport Protocol, Unsubscribe Ack
  Header Flags: 0xb0, Message Type: Unsubscribe Ack
    1011 .... = Message Type: Unsubscribe Ack (11)
    .... 0000 = Reserved: 0
  Msg Len: 2
  Message Identifier: 18026
0000 02 00 00 00 45 00 00 38 00 00 40 00 00 06 00 00  ....E..8..@...
0010 7f 00 00 01 7f 00 00 01 07 5b c1 23 90 eb 7f 32  .......[.#...2
0020 88 c7 1f 5f 80 18 18 ec fe 2c 00 00 01 01 08 0a  ...-.....*.....
0030 38 25 57 0b f0 d5 bc 40 b0 02 46 6a                8%W....@..Fj
```

12. PINGREQ (心跳请求)

触发操作：客户端每隔 Keep Alive 时间（60秒）自动发送。

字段含义：

- **Message Type (12):** 报文类型，值为12代表 PINGREQ
- 无可变报头和载荷，报文长度固定为2字节
- 作用：告知服务器客户端仍然在线

```
> Frame 7443: Packet, 58 bytes on wire (464 bits), 58 bytes captured (
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 49443, Dst Port: 1883, Seq:
MQ Telemetry Transport Protocol, Ping Request
  Header Flags: 0xc0, Message Type: Ping Request
    1100 .... = Message Type: Ping Request (12)
    .... 0000 = Reserved: 0
  Msg Len: 0
0000 02 00 00 00 45 00 00 36 00 00 40 00 00 06 00 00  ....E..6..@...
0010 7f 00 00 01 7f 00 00 01 c1 23 07 5b 88 c7 1f 6b  .......#.[...k
0020 90 eb 7f 3a 80 18 18 ec fe 2a 00 00 01 01 08 0a  ...;.....*.....
0030 f0 d6 aa d1 38 25 5b 42 c0 00                      ....8%B..
```


13. PINGRESP (心跳响应)

触发操作：EMQX 收到 PINGREQ 后立即回复。

字段含义：

- Message Type (13): 报文类型，值为13代表 PINGRESP
- 无可变报头和载荷，报文长度固定为2字节
- 作用：告知客户端服务器仍然在线

> Frame 7445: Packet, 58 bytes on wire (464 bits), 58 bytes captured (	0000	02 00 00 00 45 00 00 36	00 00 40 00 40 06 00 00	E..6..@.@...
> Null/Loopback	0010	7f 00 00 01 7f 00 00 01	07 5b c1 23 90 eb 7f 3a	[.#...
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	88 c7 1f 6d 80 18 18 ec	fe 2a 00 00 01 01 08 0a	...m....*.....
> Transmission Control Protocol, Src Port: 1883, Dst Port: 49443, Seq:	0030	38 26 45 9c f0 d6 aa d1	d0 00	8&E.....
> MQ Telemetry Transport Protocol, Ping Response				
Header Flags: 0xd0, Message Type: Ping Response				
1101 = Message Type: Ping Response (13)				
.... 0000 = Reserved: 0				
Msg Len: 0				

14. DISCONNECT (断开连接)

触发操作：MQTTX 点击断开按钮。

字段含义：

- Message Type (14): 报文类型，值为14代表 DISCONNECT
- 无可变报头和载荷
- 作用：客户端主动通知服务器断开连接，服务器不回复

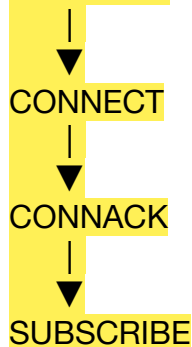
> Frame 89: Packet, 58 bytes on wire (464 bits), 58 bytes captured (46	0000	02 00 00 00 45 00 00 36	00 00 40 00 40 06 00 00	E..6..@.@...
> Null/Loopback	0010	7f 00 00 01 7f 00 00 01	c0 fd 07 5b 5f 5d 71 f3	[.]q...
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	71 ed a4 d2 80 18 18 ec	fe 2a 00 00 01 01 08 0a	q.....*.....
> Transmission Control Protocol, Src Port: 49405, Dst Port: 1883, Seq:	0030	81 0e e3 8e d0 0f 05 ea	e0 00	
> MQ Telemetry Transport Protocol, Disconnect Req				
Header Flags: 0xe0, Message Type: Disconnect Req				
1110 = Message Type: Disconnect Req (14)				
.... 0000 = Reserved: 0				
Msg Len: 0				

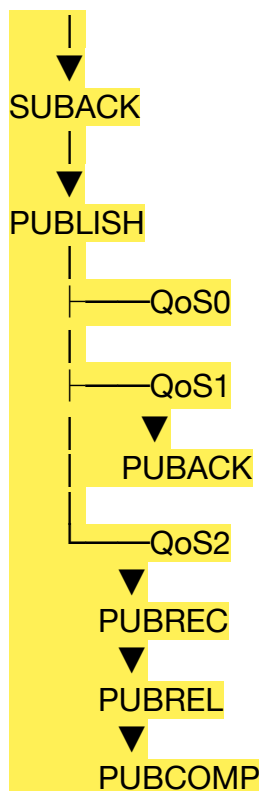
2.3 思考与总结

MQTT 协议采用发布/订阅模式，将消息的发送方（Publisher）和接收方（Subscriber）解耦，通过 Broker 进行中转，使得设备之间无需直接连接即可通信。通过本次实验，深入理解了三种 QoS 等级的区别：QoS 0 仅发送一次不确认，适合对实时性要求高但允许丢包的场景；QoS 1 通过 PUBACK 确保至少送达一次但可能重复；QoS 2 通过四步握手确保恰好送达一次，适合不允许重复的关键数据。此外，Keep Alive 机制通过 PINGREQ/PINGRESP 维持连接，避免因长时间无数据导致连接被断开。

14个报文

客户端启动





运行期间：
PINGREQ
PINGRESP

结束：
UNSUBSCRIBE
UNSUBACK

DISCONNECT

3.1 编程——python实现

实验环境：

- Python 3.9.6
- paho-mqtt 2.1.0 (通过 pip3 安装)
- EMQX 5.8.8 运行于本机 127.0.0.1:1883

实验方法：编写订阅者程序 (subscriber.py) 和发布者程序 (publisher.py)，分别实现 MQTT 消息的接收和发送功能，通过两个终端窗口同时运行，验证消息收发流程。

3.2 主要实验步骤

第一步：安装 paho-mqtt 库

```
pip3 install paho-mqtt
```

第二步：编写订阅者程序 subscriber.py，连接 EMQX，订阅主题 小鱼 和 xiaoyu，收到消息后打印输出。

```
import paho.mqtt.client as mqtt
```

```
# 连接成功时的回调函数
```

```
def on_connect(client, userdata, flags, rc, properties=None):
    print(f"连接成功, 返回码: {rc}")
    client.subscribe("小鱼", qos=1)
    client.subscribe("xiaoyu", qos=1)
    print("已订阅主题: 小鱼, xiaoyu")
```

收到消息时的回调函数

```
def on_message(client, userdata, msg):
    print(f"收到消息 - Topic: {msg.topic}, 内容: {msg.payload.decode()}")
```

```
client = mqtt.Client(client_id="paho2411630201", protocol=mqtt.MQTTv311)
client.on_connect = on_connect
client.on_message = on_message
```

```
client.connect("127.0.0.1", 1883, 60)
client.loop_forever()
EOF
```

第三步：编写发布者程序 publisher.py，连接 EMQX，分别向主题 小鱼 和 xiaoyu 发布消息。

```
import paho.mqtt.client as mqtt
import time
```

```
def on_connect(client, userdata, flags, rc, properties=None):
    print(f"连接成功, 返回码: {rc}")
```

```
client = mqtt.Client(client_id="paho2411630201pub", protocol=mqtt.MQTTv311)
client.on_connect = on_connect
```

```
client.connect("127.0.0.1", 1883, 60)
client.loop_start()
```

```
time.sleep(1)
client.publish("小鱼", payload="你好, 我是小鱼2411630201", qos=1)
print("已发送消息到主题: 小鱼")
client.publish("xiaoyu", payload="Hello, I am xiaoyu2411630201", qos=1)
print("已发送消息到主题: xiaoyu")
```

```
time.sleep(1)
client.loop_stop()
client.disconnect()
EOF
```

第四步：先运行 subscriber.py，连接成功并开始监听后，再运行 publisher.py 发送消息，订阅者终端输出收到的消息内容。

```
liuleyao@661 ~ % python3 ~/Desktop/subscriber.py
/Users/liuleyao/Desktop/subscriber.py:14: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="paho2411630201", protocol=mqtt.MQTTv311)
连接成功，返回码：0
已订阅主题：小鱼，xiaoyu
收到消息 - Topic: 小鱼，内容：你好，我是小鱼 2411630201
收到消息 - Topic: xiaoyu，内容：Hello, I am xiaoyu2411630201
[]

liuleyao ~ -zsh- 80x24
Last login: Tue Jun 23 15:00:03 on ttys008
liuleyao@661 ~ % python3 ~/Desktop/publisher.py
/Users/liuleyao/Desktop/publisher.py:7: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="paho2411630201pub", protocol=mqtt.MQTTv311)
连接成功，返回码：0
已发送消息到主题：小鱼
已发送消息到主题：xiaoyu
liuleyao@661 ~ %
```

运行结果：订阅者成功收到两条消息：

- Topic: 小鱼，内容：你好，我是小鱼2411630201
- Topic: xiaoyu，内容：Hello, I am xiaoyu2411630201

3.3 思考与总结

通过编程实验，掌握了 paho-mqtt 库的基本使用方法，理解了回调函数机制：on_connect 在连接成功时触发，on_message 在收到消息时触发。MQTT 的发布/订阅模式在编程层面体现为：发布者只需指定 Topic 和消息内容，无需关心接收方是谁；订阅者只需订阅感兴趣的 Topic，即可自动接收相关消息。这种解耦设计非常适合物联网场景中大量设备的并发通信需求。

4. Java 网络编程

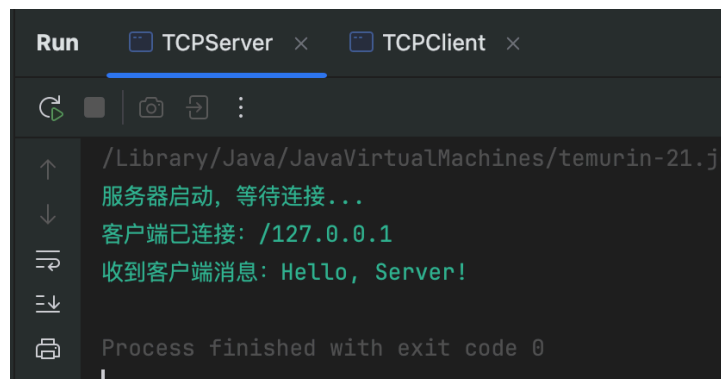
```
NetworkLab/
├── src/
│   ├── tcp/
│   │   ├── TCPServer.java
│   │   └── TCPClient.java
│   ├── udp/
│   │   ├── UDPSender.java
│   │   └── UDPReceiver.java
│   ├── multithread/
│   │   ├── MultiThreadServer.java
│   │   └── MultiThreadClient.java
│   └── mqtt/
│       └── MQTTClient.java
```

模块一：TCP 客户端与服务器通信

流程：

1. 服务器端用 ServerSocket 监听端口，等待连接
2. 客户端用 Socket 连接服务器 IP + 端口
3. 双方通过 InputStream / OutputStream 收发数据
4. 通信结束后关闭连接

关键类：ServerSocket、Socket、BufferedReader、PrintWriter



```
Run    TCPServer x    TCPClient x
/Library/Java/JavaVirtualMachines/temurin-21.j...
服务器启动，等待连接...
客户端已连接: /127.0.0.1
收到客户端消息: Hello, Server!
Process finished with exit code 0
```



```
Run    TCPServer x    TCPClient x
/Library/Java/JavaVirtualMachines/temurin-21...
收到服务器回复: 服务器已收到: Hello, Server!
Process finished with exit code 0
```

模块二：UDP 数据收发

流程：

1. 发送方创建 DatagramSocket，将数据打包成 DatagramPacket 发出
2. 接收方创建 DatagramSocket 绑定端口，调用 receive() 阻塞等待数据包
3. 解包读取内容

关键类：DatagramSocket、DatagramPacket

与 TCP 的核心区别：无连接、不保证顺序和可靠性，但速度快



```
Run    UDPReceiver x    UDPSender x
/Library/Java/JavaVirtualMachines/temurin-21...
UDP接收方启动，等待数据...
收到来自 /127.0.0.1 的消息: Hello, UDP!
Process finished with exit code 0
```

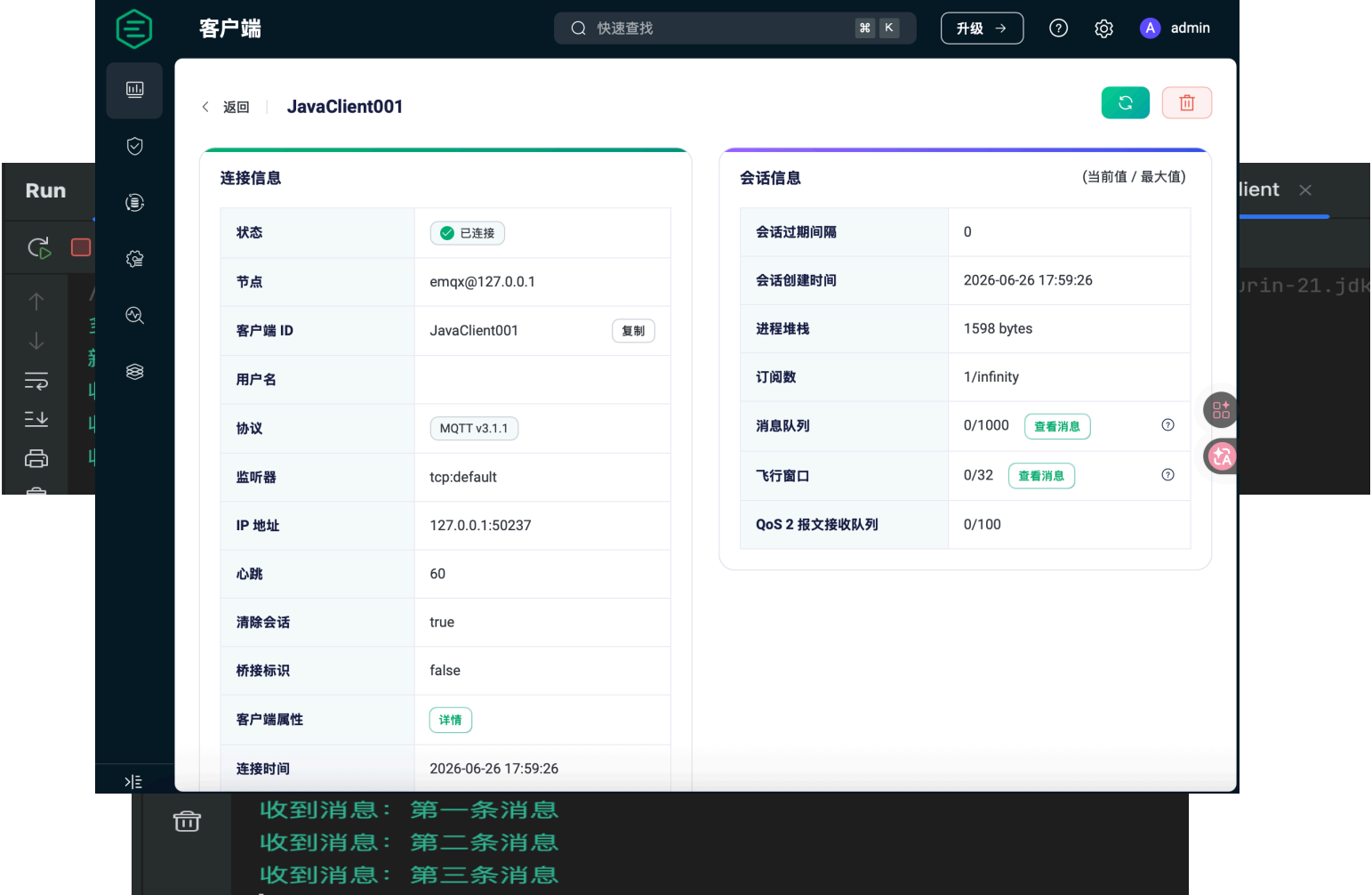


```
Run    UDPReceiver x    UDPSender x
/Library/Java/JavaVirtualMachines/temurin-21...
消息已发送: Hello, UDP!
Process finished with exit code 0
```

模块三：多线程 TCP 服务器

在模块一的基础上改造：

1. 主线程只负责 accept() 接受新连接
2. 每接受一个客户端，立即开一个新线程（Thread 或 Runnable）去处理该客户端
3. 多个客户端可以同时连接、同时通信



关键类：Thread、Runnable（或 ExecutorService 线程池）

模块四：基于 Socket 手动实现 MQTT 客户端

这是难点，不能用第三方库，需要手动拼装二进制报文：

步骤	报文类型	说明
1	CONNECT	建立 MQTT 会话，发送客户端 ID、用户名密码等
2	等待 CONNACK	服务器确认连接
3	SUBSCRIBE	订阅某个 Topic
4	等待 SUBACK	服务器确认订阅
5	PUBLISH	向某个 Topic 发布消息
6	接收 PUBLISH	收到订阅 Topic 的消息

目标服务器：EMQX 公共 Broker（broker.emqx.io:1883）

主题	QoS	操作
test/topic	0	取消订阅

客户端

快速查找

升级

admin

SubscriberClient

连接信息

状态	已连接
节点	emqx@127.0.0.1
客户端 ID	SubscriberClient
用户名	
协议	MQTT v3.1.1
监听器	tcp:default
IP 地址	127.0.0.1:52406
心跳	60
清除会话	true
桥接标识	false
客户端属性	
连接时间	2026-06-26 18:09:53

会话信息 (当前值 / 最大值)

会话过期间隔	0
会话创建时间	2026-06-26 18:09:53
进程堆栈	610 bytes
订阅数	1/infinity
消息队列	0/1000
飞行窗口	0/32
QoS 2 报文接收队列	0/100

Run MQTTClient

已连接到EMQX服务器
CONNECT已发送
CONNACK原始字节: 32 2 0 0
CONNACK响应码: 0
SUBSCRIBE已发送, Topic: test/topic
SUBACK收到, 订阅成功
PUBLISH已发送
收到消息: Hello MQTT from Java!

Process finished with exit code 0

Topic: test/topic QoS: 0

Hello MQTT from Java!

2026-06-26 18:02:13:547

5.MQTT客户端开发

使用第三方库mqttv3.jar

MQTTPublisher

发布端已连接, 请输入消息 (输入exit退出):
你好! 我是发布端
已发布: 你好! 我是发布端
exit
发布端已断开
Process finished with exit code 0

MQTTSubscriber

订阅端已连接
已订阅主题: myMqtt/test, 等待消息...
收到消息 [myMqtt/test]: 你好! 我是发布端

Topic: myMqtt/test QoS: 0

你好! 我是发布端!

2026-06-26 18:16:16:269

主题	QoS	操作
myMqtt/test	0	<button>取消订阅</button>

6.物联网应用协议设计与网络应用开发

1. 某物联网平台需要各类传感器信息，该平台设计了一个通信协议（私有 HEX），请分析理解该物联网应用协议。

该协议是北京奥尔斯教育科技（Ours）为其 IOTV3S 物联网实验平台设计的私有二进制通信协议，主要用于各类传感器节点通过网关与上层云平台进行数据交互。底层采用 MQTT 作为传输协议，应用层则使用自定义的 HEX 二进制格式。

一、协议基本结构

所有数据帧都以固定格式组织，结构简洁清晰：

- 包头：固定为 0x55
- 包长度：1字节（最大不超过 128 字节）
- 消息类型：2字节（最核心的两个是 0x0001 和 0x8001）
- 节点类型：1字节，用于区分通信方式（1=ZigBee、2=WiFi、3=EnOcean、4=Bluetooth、5=CUSTOM）
- 数据域：变长，主要承载实际业务数据
- 校验位：1字节，采用异或校验（除包头和校验位本身外所有字节异或）

协议统一使用小端模式（LSB），即低字节在前。

二、核心消息类型

该协议主要围绕两个核心消息展开：

1. 节点主动上报消息（0x0001）

这是平台中最常用的上行消息。节点默认每5秒自动上报一次（周期可通过下行命令修改）。

上报内容主要包括：

- 节点地址（长度随节点类型变化：ZigBee为2字节短地址，WiFi为4字节IP，Bluetooth为6字节MAC等）
- 3个RGB LED当前状态
- 按键及摇杆状态
- 扩展传感器数据（最重要部分）

传感器数据采用“传感器ID（2字节）+ 实际数据”的方式组织。每个传感器都有固定的ID（如温湿度是 0x0101，光照是 0x0102，三轴加速度是 0x0103 等），后面紧跟对应格式的数据（float 或整数）。

2. 云平台下发控制命令（0x8001）

这是下行核心命令，用于云平台通过网关对节点进行远程控制。主要可控制以下内容：

- 控制3个RGB LED的亮灭
- 修改节点主动上报的周期（范围1000~15000毫秒）
- 控制扩展继电器模块（常开或常闭）
- 控制电压输出模块（0~10V模拟量输出）
- 重启节点等

三、协议设计特点

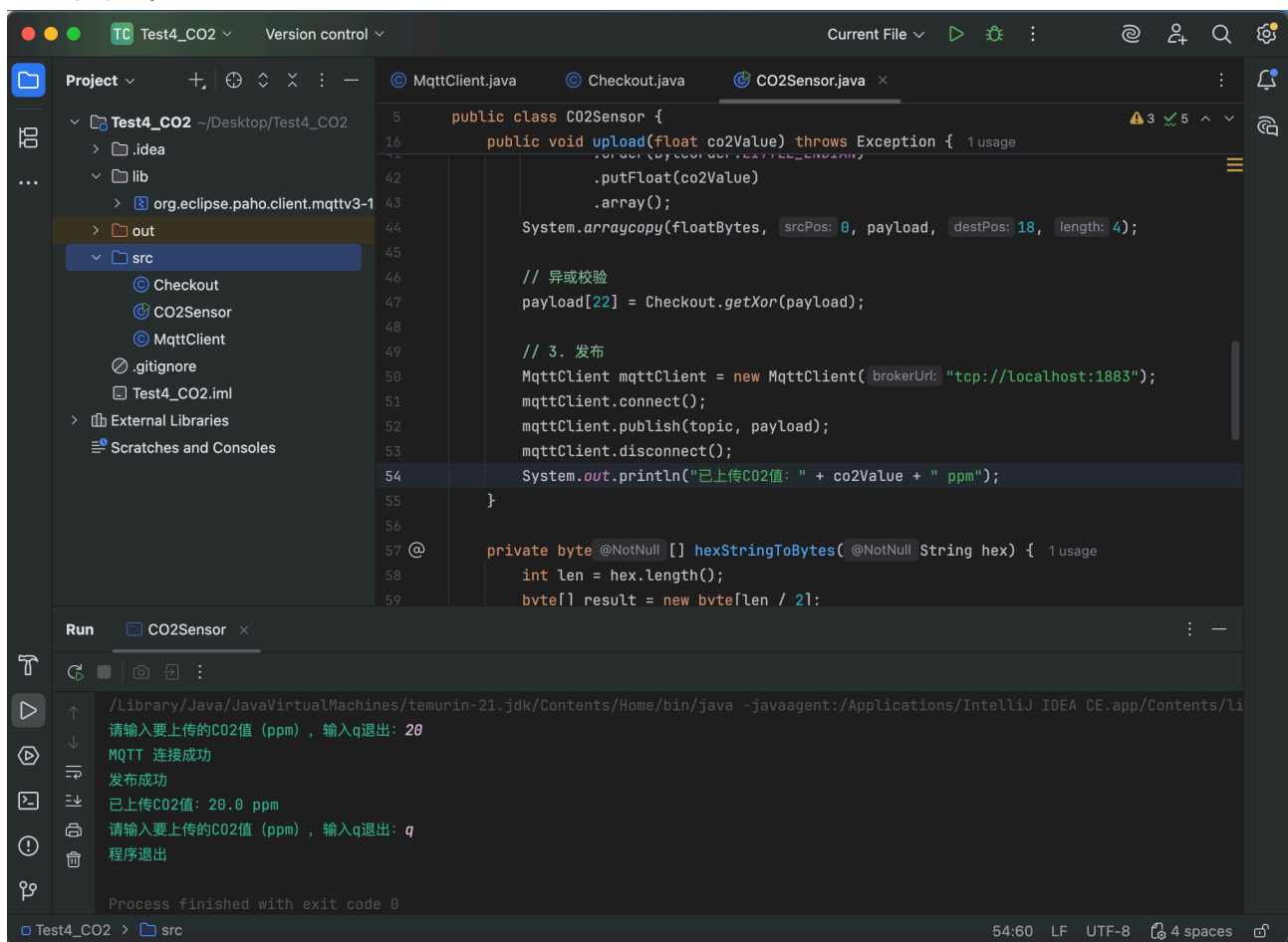
- 兼容多种通信方式：同一套协议可以同时管理 ZigBee、WiFi、EnOcean、Bluetooth 以及自定义串口节点，网关起到统一汇聚作用。
- 扩展性较好：通过定义不同的传感器ID，可以很方便地增加新的传感器类型。
- 数据紧凑：采用二进制格式，传输效率较高，适合物联网低带宽场景。
- 校验简单：仅使用单字节异或校验，适合教学和实验环境，但在工业级应用中抗干扰能力较弱。

总结

这个协议整体设计思路清晰，以 0x0001（节点状态+传感器数据主动上报）和 0x8001（云平台下发控制）作为整个系统的核心，重点围绕“状态上报 + 远程控制 + 传感器数据采集”三个场景展开。

协议更偏向于教学实验平台使用，结构不算复杂，但功能覆盖较全，适合需要同时支持多种无线协议和丰富传感器类型的教学或快速验证项目。

2.设计开发二氧化碳传感器模拟程序（数据上行），利用通信协议发送传感数据。（光照模拟器供参考）



The screenshot displays an IDE window for a project named 'Test4_CO2'. The main editor shows the 'CO2Sensor.java' file with the following code:

```
5 public class CO2Sensor {
16     public void upload(float co2Value) throws Exception { 1 usage
42         // 1. 数据封装
43         .putFloat(co2Value)
44         .array();
45         System.arraycopy(floatBytes, srcPos: 0, payload, destPos: 18, length: 4);
46
47         // 异或校验
48         payload[22] = Checkout.getXor(payload);
49
50         // 3. 发布
51         MqttClient mqttClient = new MqttClient( brokerUrl: "tcp://localhost:1883");
52         mqttClient.connect();
53         mqttClient.publish(topic, payload);
54         mqttClient.disconnect();
55         System.out.println("已上传CO2值: " + co2Value + " ppm");
56     }
57
58     private byte @NotNull [] hexStringToBytes( @NotNull String hex) { 1 usage
59         int len = hex.length();
60         byte[] result = new byte[len / 2];
```

The 'Run' console at the bottom shows the execution output:

```
/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/li
请输入要上传的CO2值 (ppm)，输入q退出: 20
MQTT 连接成功
发布成功
已上传CO2值: 20.0 ppm
请输入要上传的CO2值 (ppm)，输入q退出: q
程序退出
Process finished with exit code 0
```

Connections

test@localhost:1883

CO2@localhost:1883

+ New Subscription

IOTV3-GW/# QoS 0

Hex

All | Received | Published

5517 0100 0412 3456 abcd ef00 0000 0000
0301 0000 a041 5d
2026-06-25 14:47:22:740

Topic: IOTV3-GW/GW{1A2B3C4D5E6F} QoS: 0
5517 0100 0412 3456 abcd ef00 0000 0000
0301 0000 a041 5d
2026-06-25 14:56:31:139

Topic: IOTV3-GW/GW{1A2B3C4D5E6F} QoS: 0
U 4V0000 0A]
2026-06-25 14:56:45:369

Topic: IOTV3-GW/GW{1A2B3C4D5E6F} QoS: 0
5517 0100 0412 3456 abcd ef00 0000 0000
0301 0000 a041 5d
2026-06-25 14:58:38:998

JSON QoS 2 Retain Meta 4/4

liuleyao

```
{  
  "msg": "hi"  
}
```

客户端

快速查找

升级

admin

返回 client2411630201

刷新 删除

连接信息

状态	已连接
节点	emqx@127.0.0.1
客户端 ID	client2411630201 复制
用户名	
协议	MQTT v5.0
监听器	tcp:default
IP 地址	127.0.0.1:64905
心跳	60
Clean Start	true
桥接标识	false
客户端属性	详情
连接时间	2026-06-25 14:43:12

会话信息 (当前值 / 最大值)

会话过期间隔	0
会话创建时间	2026-06-25 14:43:12
进程堆栈	610 bytes
订阅数	1/infinity
消息队列	0/1000 查看消息
飞行窗口	0/32 查看消息
QoS 2 报文接收队列	0/100

Arduino IDE

当前订阅

刷新

+ 添加订阅

主题	QoS	禁止本地转发	发布时状态保留	保留消息处理	操作
IOTV3-GW/#	0	False	False	0	取消订阅

订阅端需要按照同样的协议格式解析字节数据，才能还原出传感器数值。

3.设计开发一个继电器模拟程序（指令下行），利用通信协议发送传感数据。

RelaySensor 启动

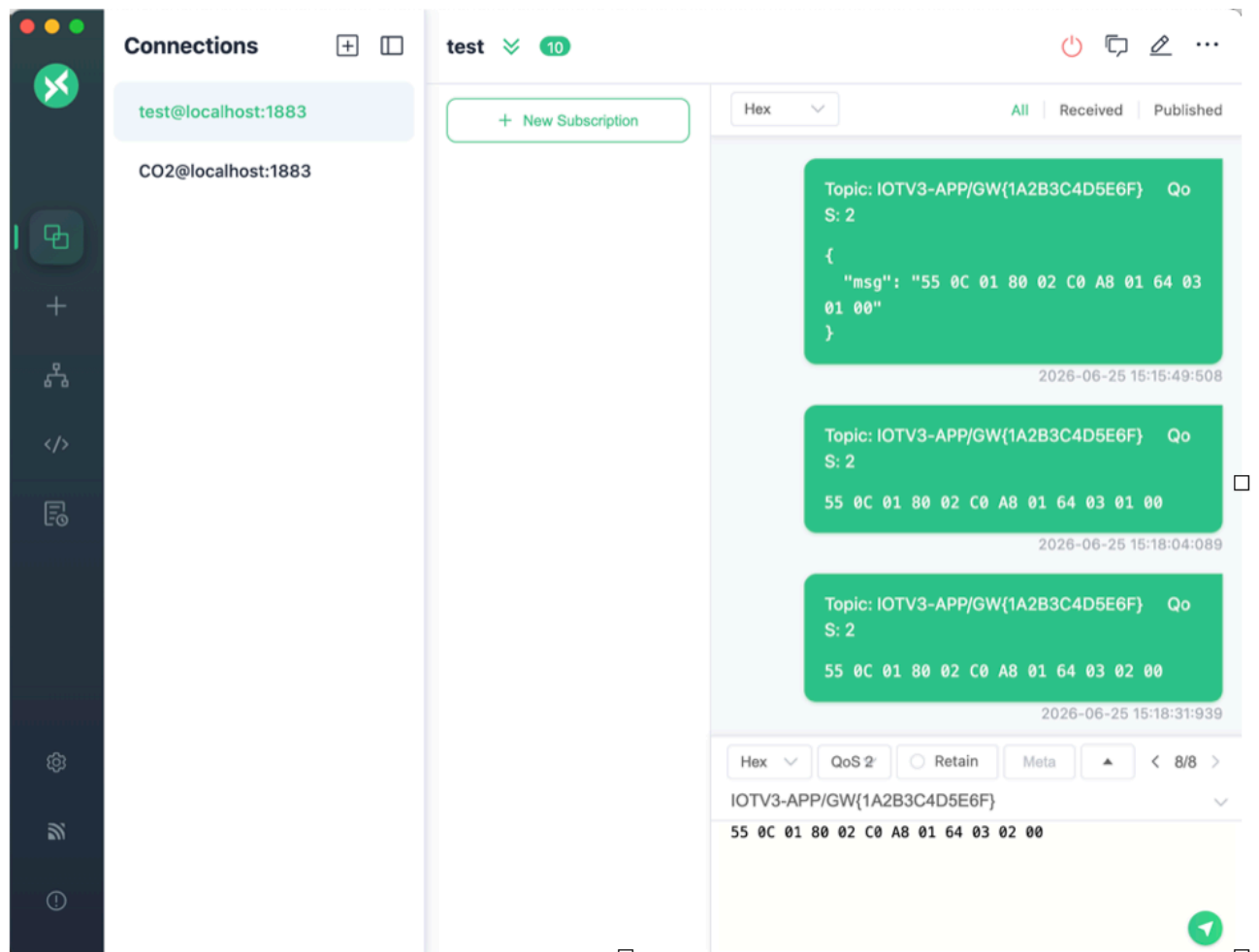
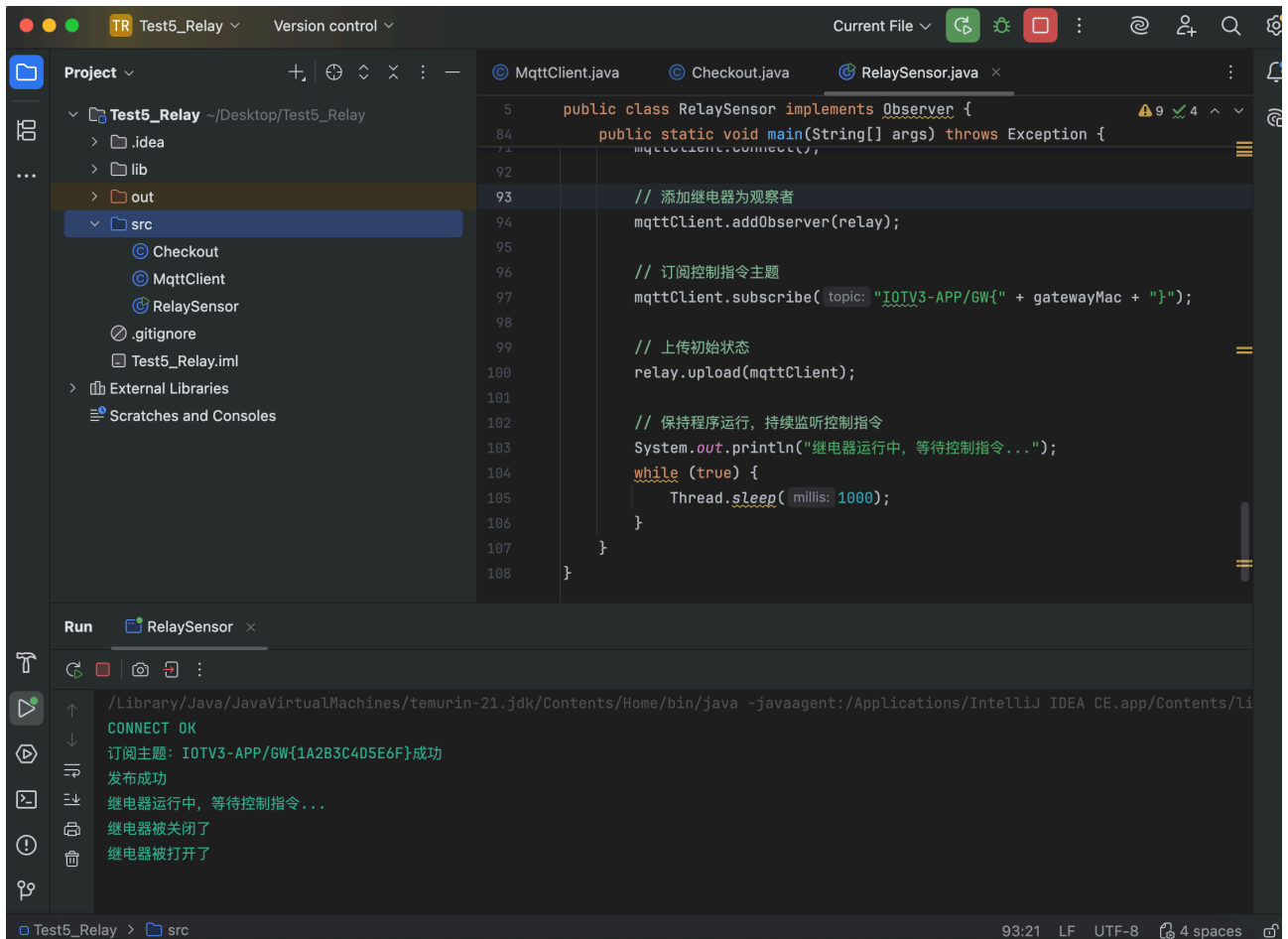
- ↓ 上行：发布继电器状态到 IOTV3-GW/GW{MAC}
- ↓ 同时订阅：IOTV3-APP/GW{MAC} 等待控制指令
- ↓ 收到指令后解析第10字节
- ↓ 0x01=关闭继电器，0x02=打开继电器

关闭指令：55 0C 01 80 02 C0 A8 01 64 03 01 00

打开指令：55 0C 01 80 02 C0 A8 01 64 03 02 00

均需要选择hex

字节	值	含义
0	55	包头
1	0C	包长度12
2-3	01 80	控制指令类型
4	02	Wifi节点
5-8	C0 A8 01 64	192.168.1.100
9	03	控制对象
10	01/02	关闭/打开
11	00	校验（暂时填0测试）



客户端

快速查找

升级

admin

返回

RelaySensor_001

刷新

删除

连接信息

状态	已连接
节点	emqx@127.0.0.1
客户端 ID	RelaySensor_001
用户名	
协议	MQTT v3.1.1
监听器	tcp.default
IP 地址	127.0.0.1:49410
心跳	60
清除会话	true
桥接标识	false
客户端属性	详情
连接时间	2026-06-25 15:12:17

会话信息

(当前值 / 最大值)

会话过期间隔	0
会话创建时间	2026-06-25 15:12:17
进程堆栈	987 bytes
订阅数	1/infinity
消息队列	0/1000
飞行窗口	0/32
QoS 2 报文接收队列	0/100

当前订阅

刷新

添加订阅

主题	QoS	操作
IOTV3-APP/GW{1A2B3C4D5E6F}	1	取消订阅

4.结合第2步和第3步的程序，实现智能家居应用的以下功能

更新协议，实现一个PM2.5 传感器

如果 室内 CO2 浓度高于 1000 ppm 且 室外 PM2.5 低于 35 $\mu\text{g}/\text{m}^3$ ，则通过继电器开启新风系统。不满足条件时关闭。

（可选）通过另一个继电器打开智能窗户。

联动过程需要经过MQTT代理中转，程序完成通信协议解析。

（可选）设立遗嘱消息，传感器意外断开n秒后，可以关闭继电器。

CO2Sensor → 发布CO2数据 → EMQX

PM25Sensor → 发布PM2.5数据 → EMQX

↓

Controller（核心）← 订阅两个传感器数据，判断逻辑

↓

发布控制指令



RelaySensor (新风) ← 订阅指令, 执行开关

RelaySensor (窗户) ← 订阅指令, 执行开关

对, `co2Values` 和 `pm25Values` 数组里的数据是我写死的测试数据, 用来验证联动逻辑是否正确:

轮次	CO2	PM2.5	结果
1	800	20	关闭 (CO2不超标)
2	1200	25	开启 (两个条件都满足)
3	1100	40	CO2收到时开启, PM2.5收到后关闭 (PM2.5超标)
4	900	30	关闭 (CO2不超标)

条件是: $\text{CO}_2 > 1000 \text{ ppm}$ 且 $\text{PM}_{2.5} < 35 \mu\text{g}/\text{m}^3$

意思是: 室内CO2超标, 同时室外空气质量好, 才开窗换气。如果室外PM2.5也超标就不开, 开了反而更糟。

```
public class Controller implements Observer {
    private void checkAndControl() {
        windowRelay.sendControl(mqttClient, open: true);
    } else {
        System.out.println("【联动触发】条件不满足 (CO2=" + co2Value + "ppm, PM2.5=" + pm25Value +
        ventRelay.sendControl(mqttClient, open: false);
        windowRelay.sendControl(mqttClient, open: false);
    }
    String status = "CO2=" + co2Value + "ppm | PM2.5=" + pm25Value + "μg/m³ | 新风=" + (shouldOpen ? "开启" : "关闭");
}
```

Run Controller

```
/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ
【SmartHome_Controller】连接成功
订阅主题: IOTV3-GW/GW{1A2B3C4D5E6F}成功
【CO2Sensor_001】连接成功
【PM25Sensor_001】连接成功
智能家居系统启动, 每5秒上传一次数据...

--- 第1轮数据 ---
CO2数据已上传: 800.0 ppm
收到室内CO2: 800.0 ppm
PM2.5数据已上传: 20.0 μg/m³
收到室外PM2.5: 20.0 μg/m³
【联动触发】条件不满足 (CO2=800.0ppm, PM2.5=20.0μg/m³) → 关闭新风和窗户
【新风系统】继电器指令: 关闭
【智能窗户】继电器指令: 关闭

--- 第2轮数据 ---
CO2数据已上传: 1200.0 ppm
收到室内CO2: 1200.0 ppm
PM2.5数据已上传: 25.0 μg/m³
收到室外PM2.5: 25.0 μg/m³
```

```
Run Controller x
--- 第2轮数据 ---
C02数据已上传: 1200.0 ppm
收到室内C02: 1200.0 ppm
PM2.5数据已上传: 25.0 µg/m³
收到室外PM2.5: 25.0 µg/m³
【联动触发】C02=1200.0ppm, PM2.5=25.0µg/m³达标 → 开启新风和窗户
【新风系统】继电器指令: 打开
【智能窗户】继电器指令: 打开

--- 第3轮数据 ---
C02数据已上传: 1100.0 ppm
收到室内C02: 1100.0 ppm
PM2.5数据已上传: 40.0 µg/m³
收到室外PM2.5: 40.0 µg/m³
【联动触发】条件不满足 (C02=1100.0ppm, PM2.5=40.0µg/m³) → 关闭新风和窗户
【新风系统】继电器指令: 关闭
【智能窗户】继电器指令: 关闭

--- 第4轮数据 ---
C02数据已上传: 900.0 ppm
收到室内C02: 900.0 ppm
PM2.5数据已上传: 30.0 µg/m³
收到室外PM2.5: 30.0 µg/m³

✅ 所有数据发送完毕, 系统保持运行中
按回车键退出...
```

6_SmartHome > src > Controller > checkAndControl 79:42 LF UTF-8 4 spaces

test@localhost:1883

CO2@localhost:1883

test 103

+ New Subscription

IOTV3-GW/GW{...} QoS 0

IOTV3-APP/GW{...} QoS 0

SmartHome/Stat... QoS 0

Plaintext

All Received Published

Topic: IOTV3-APP/GW{1A2B3C4D5E6F} QoS: 0

U 000d 0

2026-06-25 16:27:28:615

Topic: SmartHome/Status QoS: 0

C02=1100.0ppm | PM2.5=40.0µg/m³ | 新风=关闭 | 窗户=关闭

2026-06-25 16:27:28:616

Topic: IOTV3-GW/GW{1A2B3C4D5E6F} QoS: 0

U 4V000 aD0

2026-06-25 16:27:33:606

Topic: IOTV3-GW/GW{1A2B3C4D5E6F} QoS: 0

U 4V000 0A

2026-06-25 16:27:33:636

Hex QoS 2 Retain Meta 8/8

Topic

客户端

快速查找

升级

admin

返回

client2411630201

刷新

删除

连接信息

状态	已连接
节点	emqx@127.0.0.1
客户端 ID	client2411630201
用户名	
协议	MQTT v3.1.1
监听器	tcp:default
IP 地址	127.0.0.1:50228
心跳	60
清除会话	true
桥接标识	false
客户端属性	详情
连接时间	2026-06-25 16:14:46

会话信息

(当前值 / 最大值)

会话过期间隔	0
会话创建时间	2026-06-25 16:14:46
进程堆栈	987 bytes
订阅数	3/infinity
消息队列	0/1000
飞行窗口	0/32
QoS 2 报文接收队列	0/100

当前订阅

刷新

+ 添加订阅

主题	QoS	操作
IOTV3-GW/GW{1A2B3C4D5E6F}	0	取消订阅
IOTV3-APP/GW{1A2B3C4D5E6F}	0	取消订阅
SmartHome/Status	0	取消订阅